

ENTERPRISE AI KNOWLEDGE ASSISTANT

Enterprise Project Management Toolkit

Technical Design Document

System Architecture – Enterprise AI Knowledge Assistant

Document Title	Technical Design Document (TDD)
Project Name	Enterprise AI Knowledge Assistant (EAIKA)
Author	Engineering / Architecture Team
Version	1
Status	Approved for Build
Date	4-Jul-26

1. Overview

The Enterprise AI Knowledge Assistant (EAIKA) is a Retrieval-Augmented Generation (RAG) application implemented in Python. It combines a document processing pipeline, a vector-based semantic retrieval layer, and a locally hosted large language model to answer natural-language questions over enterprise documents, exposed through a Streamlit web application. The system is designed to run entirely within the organization's own infrastructure, with no dependency on external cloud AI APIs.

2. Architecture Principles

- Local-first: all inference and vector storage run on infrastructure controlled by the organization.
- Modularity: ingestion, chunking, embedding, vector storage, retrieval, and generation are implemented as independent services that can be modified or replaced independently.
- Transparency: every generated answer must be traceable to the specific source chunk(s) it was derived from.
- Extensibility: the same pipeline architecture supports additional document domains beyond the initial project-management pilot.

3. High-Level Component Architecture

The codebase is organized into a Streamlit entry point (app.py), a components layer for UI rendering, and a services layer for the core RAG pipeline logic, consistent with a modular Python application layout.

Layer / Folder	Responsibility
app.py	Streamlit application entry point; wires together UI components and backend services; defines page routing between the chat interface and administrator dashboard.
components/	Reusable Streamlit UI components — chat window, message rendering, source citation display, file uploader, admin statistics panels.
services/	Core backend logic — document loading and text extraction, chunking, embedding generation, vector store interface (ChromaDB), retrieval logic, and LLM orchestration via LangChain/Ollama.
data/documents/	Storage location for uploaded source documents prior to and after processing.
chroma_db/	Persisted ChromaDB vector store containing chunk embeddings and associated metadata.
requirements.txt	Python dependency manifest (LangChain, Streamlit, ChromaDB client, OCR and document-parsing libraries, Ollama client).

4. Technology Stack

Layer	Technology	Purpose
UI / Presentation	Streamlit	Web-based chat interface and administrator dashboard, rendered as a single Python web app.
Orchestration	LangChain	Chains together document loaders, text splitters, retrievers, and the LLM call into a single RAG pipeline.
Language Model	Qwen2.5 (served via Ollama)	Locally hosted LLM used to generate natural-language answers grounded in retrieved context.
LLM Runtime	Ollama	Local model server that hosts and serves the Qwen2.5 model without external API calls.
Vector Database	ChromaDB	Stores chunk embeddings and metadata; performs similarity search for retrieval.
Document Parsing	Format-specific parsers	Extracts text from PDF, DOCX, PPTX, XLSX, TXT, CSV, and EML files.

OCR	Image/OCR engine	Extracts text from PNG/JPG/JPEG images and scanned or image-based PDF pages.
Language / Runtime	Python 3.11+	Primary implementation language for the entire application.

5. End-to-End Data Flow

5.1 Ingestion Flow

1. An administrator uploads one or more documents through the Administrator Dashboard (components/ upload widget).
2. The relevant document loader in services/ extracts raw text based on file type; OCR is invoked automatically for image-based content.
3. Extracted text is split into overlapping chunks sized to balance retrieval precision and context completeness.
4. Each chunk is passed through the embedding model to generate a vector representation.
5. Chunk vectors, chunk text, and source metadata (file name, location, section) are persisted to the ChromaDB collection under chroma_db/.
6. Knowledge base statistics (document count, chunk count, last build time) are updated and surfaced on the Administrator Dashboard.

5.2 Query / Answer Flow

7. A user submits a natural-language question through the Streamlit chat interface.
8. The question is embedded using the same embedding model used during ingestion, ensuring vector-space consistency.
9. ChromaDB performs a similarity search and returns the top-N most relevant chunks.
10. LangChain assembles a prompt containing the retrieved chunks and the user's question and sends it to the Qwen2.5 model via Ollama.
11. The model generates a natural-language answer grounded in the retrieved context.
12. The application returns the answer to the user along with the source document reference(s) used to generate it.

6. Component Interaction Diagram (Textual)

User Browser → Streamlit UI (app.py / components) → Retrieval Service (services/) → ChromaDB (vector search) → LangChain Orchestrator → Ollama / Qwen2.5 (answer generation) → Response + Source Citations → Streamlit UI → User Browser. The Administrator Dashboard interacts with the Ingestion Service and ChromaDB directly to manage the knowledge base independently of the query flow.

7. Deployment Architecture

- EAIKA is designed for on-premises or private-cloud (VPC) deployment on a single application host or small cluster, with no external network dependency for AI inference. A representative deployment consists of:
- An application server running the Streamlit process (app.py) and the services layer.
 - A local Ollama instance hosting the Qwen2.5 model, either co-located with the application server or on a dedicated GPU/CPU host reachable over the internal network.
 - A persistent volume for the ChromaDB vector store (chroma_db/) and the raw document store (data/documents/), backed up on the organization's standard backup schedule.
 - Standard enterprise network controls (firewall, internal DNS, TLS termination at a reverse proxy) in front of the Streamlit application.

8. Security & Data Protection Design

- No document content or user query is transmitted to an external/cloud AI API; all inference occurs via the local Ollama-hosted model.
- Access to the application should be placed behind the organization's existing authentication layer (e.g., SSO/reverse proxy) since the base application does not include its own identity provider.
- Uploaded documents and the vector store should reside on encrypted storage volumes consistent with the organization's data-at-rest policy.
- Administrator functions (upload, rebuild) should be restricted to a defined administrator group at the network or reverse-proxy layer.

Detailed risk treatment is documented in the AI Risk Evaluation & Safety Case and the Enterprise Readiness & SLA document.

9. Scalability & Performance Considerations

- Chunking parameters (chunk size / overlap) directly affect both retrieval accuracy and vector store size; these should be tuned per document domain during the build phase.
- ChromaDB performance scales with available memory and disk I/O; capacity planning is addressed in the Infrastructure & Capacity Plan.
- LLM response latency is primarily a function of the Ollama host's compute resources (CPU/GPU) and the Qwen2.5 model size; hardware sizing is addressed in the Infrastructure & Capacity Plan.
- The knowledge base build/rebuild operation is I/O and compute intensive and should be scheduled during low-usage windows for large document sets.

10. Error Handling & Resilience

- Document parsing failures (corrupt files, unsupported sub-formats) are surfaced to the administrator rather than silently skipped.
- Queries that return no relevant retrieval results produce an explicit “no relevant information found” response rather than a fabricated answer.
- The application should implement retry logic for transient failures when calling the local Ollama service, with a user-facing timeout message if the model does not respond within an acceptable window.

11. Future Architecture Considerations

- Introduce role-based access control and per-document permissions ahead of expanding into HR, Finance, and Healthcare domains.
- Add an audit log service to record which user asked which question and which sources were returned, supporting compliance requirements in later phases.
- Evaluate a message-queue-based ingestion pipeline to support larger-scale, asynchronous document processing as the knowledge base grows.

12. Approval

This Technical Design Document requires review and sign-off by the Engineering Lead, Architecture Review Board, and Information Security Officer prior to development commencing.